



# Database Driven Web Application

COMP1048: Databases and Interfaces (2025-2026)

Matthew Pike & Yanwen Mao

## Table of contents

|                                                         |    |
|---------------------------------------------------------|----|
| Document Revision History                               | 1  |
| 1 Overview                                              | 2  |
| 2 Tasks                                                 | 3  |
| 2.1 Task 1: Upload and Update Playlist Tables (5 Marks) | 3  |
| 2.2 Task 2: Generate Playlist Statistics (8 Marks)      | 5  |
| 2.3 Task 3: Playlist CRUD Operations (12 Marks)         | 8  |
| 3 Submission                                            | 12 |
| 3.1 Penalties                                           | 12 |
| 3.2 Academic Integrity                                  | 12 |
| Acknowledgements                                        | 13 |

## Document Revision History

We will maintain a record of changes to this document here.

| Version | Date       | Description     |
|---------|------------|-----------------|
| 1.0     | 2025-12-01 | Initial Version |

Table 1: Document Revision History.



# 1 Overview

## **i** The iMusic Scenario is Fictitious

Please note that this assignment is based on a fictional scenario, and the company mentioned does not exist. Furthermore, the data provided may not be accurate or complete.

iMusic is a company that sells music. Previously, the company operated a physical store in the city centre but has recently transitioned to an online-only business model. To support this change, they plan to update their website.

iMusic initially hired a professional developer to carry out this work, but due to other commitments, the developer was unable to complete it. You have now been hired to finish the project.

iMusic has provided a list of website requirements outlined in the following sections. The website is partially implemented, and your task is to complete its development. To complete the assignment, you will need to modify only the `iMusic.py` file. The starting implementation is provided in the `code/` directory. The website uses the following technologies:

- **HTML + Jinja2**: For the website's front-end. The previous developer has already created the necessary web pages using HTML and Jinja2 templates. **You should not modify these templates.** **Bootstrap 5** has been used for styling.
- **Python + Flask**: For the website's back-end. The website is built using the Flask web application framework. You will need to complete the Python functions in the `iMusic.py` file to implement the required functionality. The iMusic team use Python 3.12. During testing of your solution, the iMusic team will use the 3.0.3 version of Flask.
- **SQLite**: For the website's database. The website stores its data in an SQLite database. You will need to interact with the database using the `sqlite3` module in Python. Do not use any other database libraries or modules such as SQLAlchemy or Pandas. During the testing of your solution, the iMusic team will use the 3.44.0 version of SQLite.
  - Please note: The database schema is identical to the database used in the DBI Quiz. Some of the data, however, has been modified for this assignment. The original source of the database is the **Chinook Database**. Please note that you are expected to familiarise yourself with the database schema and the data/relationships using the skills you have learned in this module. The iMusic team will not provide any additional documentation on the database schema.

**You must only use modules already imported in `iMusic.py` or modules from the Python standard library. No other imports are allowed.**

## **i** Important Terminology Note

In this assignment you might be working with a few terms that are unfamiliar or have different meanings in other contexts. Here are some clarifications:

- **Track** is equivalent to what might commonly be called a “song” in other contexts.
- **Playlist** refers to a collection of tracks grouped together by a user, often for a specific mood or activity.
- **Genre** refers to the category or style of music (e.g., Rock, Jazz, Classical).



## 2 Tasks

### 2.1 Task 1: Upload and Update Playlist Tables (5 Marks)

---

#### 2.1.1 Task Brief

The iMusic team wants to allow users to upload playlist track data to update their playlist database. You have been provided with an example **Tab-separated values (TSV)** file containing playlist-track relationships, which you will use to update the database. To do this, modify the `update_playlist_tracks` function in the `iMusic.py` file. This function will be automatically triggered when a user uploads a TSV file via the form on the `/upload/` route of the website.

The uploaded file will be a TSV file containing the following columns:

- `PlaylistId`: The ID of the playlist to be updated (**required**).
- `TrackId`: The ID of the track to be added to the playlist (**required**).

Both `PlaylistId` and `TrackId` fields will always be provided in the TSV file. You must validate that both the playlist and track exist in the database before inserting the record into the `PlaylistTrack` table. The provided TSV file will always have a header row with the column names as shown above.

To parse the TSV file, you must use the `csv` module, which is part of the Python standard library. The `csv` module provides functionality for reading and writing CSV and TSV files. You can refer to the [csv module documentation](#) for more information. Reading and comprehending the documentation is an essential skill for developers, and an assessed skill in this assignment.

Ensure that each `PlaylistId` and `TrackId` exists in the database before inserting. If either the playlist or track does not exist in the database, ignore that record in the TSV file and continue processing the next record. You should also avoid inserting duplicate combinations that already exist in the `PlaylistTrack` table. If duplicates are found, ignore them and continue processing the next record.

Once the playlist track data has been updated, the user will be redirected back to the index page (`/`). The `upload_route` function already includes error handling. If your `update_playlist_tracks` function encounters an error, the `upload_route` will catch it, display an error message, and redirect the user back to the upload page. `update_playlist_tracks` should return `True` on successful completion (i.e., processed without fatal errors). On success, the user will be redirected to the index page. Your solution should handle any errors gracefully and not crash the server.

#### 2.1.2 Evaluation Procedure

When testing your solution, the iMusic team will:

1. Navigate to the `/upload/` route on the website.
2. Upload a TSV file containing playlist-track data. Several TSV files with different playlist-track combinations will be used in testing; these files will not be provided to you in advance. Ensure your solution can handle any valid TSV file, not just the example provided. You are encouraged to test your solution with different TSV files you create to ensure it works correctly.
3. Submit the form to trigger the `update_playlist_tracks` function.



4. Verify that the `PlaylistTrack` table in the database has been correctly updated with only valid combinations.

### 2.1.3 Constraints

Your implementation must meet the following requirements:

- The entire implementation must be contained within the `update_playlist_tracks` function. Do not create additional functions or modify the template file.
- The implementation must run as a Flask application, using the command `python iMusic.py` or `python3 iMusic.py`.
- Use the `sqlite3` module exclusively for database interactions. Other modules or approaches (e.g., `SQLAlchemy`) are not permitted.
- Use the `csv` module for reading the TSV file. Refer to the `csv` module documentation as needed. Other modules or approaches (e.g., `Pandas`) are not allowed.

### 2.1.4 Marking Scheme

Task 1 is worth 5 of the 25 marks available for the assignment. The following criteria will be used to assess your work:

| ID    | Requirement              | Details                                                                                                                | Marks |
|-------|--------------------------|------------------------------------------------------------------------------------------------------------------------|-------|
| RQ1_1 | Correct File Reading     | Successfully reads the TSV file using the <code>csv</code> module, correctly parsing the playlist-track data.          | 0.5   |
| RQ1_2 | Database Connection      | Establishes a connection to the SQLite database using the <code>sqlite3</code> module.                                 | 0.5   |
| RQ1_3 | Correct Data Handling    | Correctly validates that both <code>PlaylistId</code> and <code>TrackId</code> exist in the database before insertion. | 2     |
| RQ1_4 | Accurate Database Update | Accurately inserts valid playlist-track combinations into the <code>PlaylistTrack</code> table without duplicates.     | 2     |

Table 2: Marking Criteria for Task 1.



## 2.2 Task 2: Generate Playlist Statistics (8 Marks)

### 2.2.1 Task Brief

The iMusic team requires an overview of playlist statistics to enhance their understanding of playlist composition across different genres. You have been tasked with implementing new functionality on the website that displays statistics for all playlists filtered by genre, with sorting capabilities. Additionally, you will need to provide the option of viewing statistics for all playlist tracks, denoted by the (non-existent) genre 'All'. To achieve this, you will need to complete the `statistics`, `get_all_genres` and `get_statistics` functions.

When a user navigates to the `/statistics/` route, the `statistics` function should be triggered. You must add the appropriate decorator to the function to define the route. This function should render the `statistics.html` template file. The template file contains a form with three dropdown menus that allow users to select a genre and sorting options (column and order). By default, 'All', 'Playlist ID', and 'ASC' should be selected in the dropdowns, respectively. Note 'All' must be spelled exactly as shown - capital 'A' and lowercase 'll'. When the user selects a genre and sorting options from the dropdowns and submits the form, the page should display the relevant statistics for each playlist based on the selected genre and sorting criteria. The selected options should persist in the dropdowns after submission. This logic is already implemented in the template file; you must examine the `statistics.html` file to identify the specific variable names expected by the template to maintain this state, and ensure you pass them correctly in your `render_template` return statement. The statistics route must handle both GET requests (initial page load) and POST requests (form submission).

You must complete the `get_all_genres` and `get_statistics` functions to support this functionality. The `get_all_genres` function should retrieve all genres from the Genre table in the database, sorted in ascending alphabetical order by genre name. Additionally, the option 'All' should be included first in the list with a `GenreId` of 0, but should not be present in the database - it is not a real genre. First retrieve all genres sorted alphabetically, then prepend the 'All' option (with `GenreId` of 0) to the beginning of the list. The retrieved genres will be used to populate the genre dropdown in the template. You must check the template to determine how to format the data correctly. The `statistics` function should call the `get_all_genres` function to retrieve the list of genres and pass this data to the template for rendering.

Upon selecting a genre and sorting options from the dropdowns, the page should display the following statistics for each playlist (you must examine the `statistics.html` file to ensure your SQL column aliases match the dictionary keys expected by the template):

- **Playlist ID:** The ID of the playlist.
- **Playlist Name:** The name of the playlist.
- **Number of Tracks:** The total number of tracks in the playlist that match the selected genre (or all tracks if 'All' is selected).
- **Duration (nearest minute):** . The total duration of matching tracks converted to minutes. The `Milliseconds` column in the `Track` table stores duration in milliseconds. You must convert to minutes using floating-point division. The template handles the final rounding for display (only). Note: Be aware that SQLite performs integer division by default; you must ensure you calculate the precise floating-point value. Test your SQL queries in SQLite and manually verify the results before implementing them in your code.
- **Total Cost:** The total cost (sum of `UnitPrice`) of all matching tracks in the playlist.
- **Average Cost:** The average cost of all matching tracks in the playlist.



- If a playlist has no matching tracks, all numerical statistics should be displayed as 0. Note: Standard SQL aggregation (e.g., SUM) returns NULL for empty datasets. You must handle this to ensure 0 is returned instead of None.

The code for retrieving and calculating these statistics should be implemented in the `get_statistics` function. This function should accept parameters for genre filtering and sorting, and return the appropriate playlist statistics based on the user's selection. Playlists should be displayed according to the selected sorting criteria (**Playlist ID**, **Name**, **Number of Tracks**, **Duration (nearest minute)**, **Total Cost**, or **Average Cost**) and order (**ASC/DESC**). The `statistics` function should call the `get_statistics` function with the selected genre and sorting options, and pass the returned statistics data to the template for rendering.

Your solution must also handle scenarios where invalid selections are provided. As a general security principle, we cannot trust any data received from the user. It is possible for a user to manipulate the HTML and send a request with values that are not present in the dropdowns or are maliciously crafted.

To ensure our application is secure and robust, you must safeguard against all user-supplied inputs:

- **Invalid Genre:** If a request is received with a `GenreId` that is not in the database (and is not 'All'/0), the page must redirect the user back to the `/statistics/` route with the message 'Invalid genre ID' and an alert type of danger.
- **Invalid Sorting:** Similarly, the `SortBy` and `SortOrder` inputs must be validated. If an invalid value is received, your application should redirect the user back to the `/statistics/` route with the message 'Invalid sorting parameters' and an alert type of danger. Hint: You'll find that SQLite does not support parameterized queries for column names or order directions. Consider using a whitelist approach to validate and map user inputs to actual column names and order directions in your SQL query.

There is no need to explicitly handle any other errors; but if you chose to, then you may redirect back to the `/statistics/` route and use the message 'An error occurred' with an alert type of danger.

## 2.2.2 Evaluation Procedure

When testing your solution, the iMusic team will:

1. Navigate to the `/statistics/` route on the website.
2. Select a genre from the dropdown menu and choose sorting options, then press Submit.
3. Verify that the playlist statistics for the selected genre are displayed correctly.
4. Verify that the sorting functionality works properly for different columns and orders.

To further test your solution, the iMusic team will add new playlists and tracks to the database and check that the statistics page updates correctly. Ensure that your solution can handle any valid data in the database, not just the examples provided. You are encouraged to test your solution with different data to ensure it works correctly — you will need to create your own test data for this task. The iMusic team will not provide you with additional test data. Similarly, the iMusic team will attempt to access the `/statistics/` route with an invalid genre selection to verify that the error message is displayed correctly.



### 2.2.3 Constraints

Your implementation must meet the following requirements:

- The entire implementation must be contained within the `statistics`, `get_all_genres`, and `get_statistics` functions. Do not create additional functions or modify the template file.
- The implementation must run as a Flask application, using the command `python iMusic.py` or `python3 iMusic.py`.
- Use the `sqlite3` module exclusively for database interactions. Other modules or approaches (e.g., `SQLAlchemy`) are not permitted.
- Do not modify the provided template file (`templates/statistics.html`).

### 2.2.4 Marking Scheme

Task 2 is worth 8 of the 25 marks available for the assignment. The following marking criteria will be used to assess your work:

| ID    | Requirement                     | Details                                                                                                                                                               | Marks |
|-------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| RQ2_1 | Genre Data Retrieval            | Correct implementation of <code>get_all_genres</code> to retrieve and return a list of genres in ascending order, with "All" appearing first.                         | 2     |
| RQ2_2 | Playlist Statistics Calculation | Accurate implementation of <code>get_statistics</code> to calculate all required playlist statistics for the selected genre.                                          | 2     |
| RQ2_3 | Dynamic Sorting Implementation  | Proper implementation of sorting within <code>get_statistics</code> , correctly applying the user-selected <code>SortBy</code> and <code>SortOrder</code> parameters. | 2     |
| RQ2_4 | Input Validation and Security   | Proper error handling for invalid data and implementation of security measures to prevent SQL injection and ensure database integrity.                                | 2     |

Table 3: Marking Criteria for Task 2.



## 2.3 Task 3: Playlist CRUD Operations (12 Marks)

### 2.3.1 Task Brief

The iMusic team wants to provide playlist management functionality for users. This task requires implementing full CRUD (Create, Read, Update, Delete) operations for playlists. You will need to complete multiple functions in the `iMusic.py` file to support playlist creation, renaming, deletion, and dynamic track management (adding/removing tracks by genre).

When a user navigates to the `/playlists/` route, the `playlists` function will be triggered. You must add the appropriate decorator to the function to define the route. This function should render the `playlists.html` template file, which provides an interface for managing playlists. The template includes forms for creating new playlists, renaming existing ones, deleting playlists, and adding/removing tracks by genre. Remember you should not modify the template file. The `playlists` function should call `get_all_playlists` and `get_all_genres` (implemented in Task 2) to retrieve the current playlist and genre data from the database and pass this data to the template for rendering. You must inspect the template to determine how to format the data correctly and what variables to pass. `get_all_playlists` should return all playlists from the `Playlist` table sorted by Name in ascending order.

The playlist management system should support the following operations via separate routes and functions in `iMusic.py`:

1. **Create Playlist:** Users can create new (empty) playlists by providing a valid name. Names do not need to be unique, but they cannot be empty.
  - **Route:** `/playlist/create/`
  - **Function:** `create_playlist` - It must validate that the playlist name is not empty before creating the playlist in the database.
  - **Error:** If the playlist name is empty, redirect back to `/playlists/` with the message 'Playlist name cannot be empty' and an alert type of danger. Note that there are no specific requirements for handling whitespace in names - this will not be tested.
  - **Success:** On successful creation, redirect back to `/playlists/` with the message 'Playlist created successfully' and an alert type of success.
2. **Rename Playlist:** Users can rename existing playlists by selecting the desired playlist and providing a new name.
  - **Route:** `/playlist/rename/`
  - **Function:** `rename_playlist` - It must validate that the playlist exists and that the new name is not empty before updating the playlist name in the database.
  - **Errors:** (If multiple errors occur, display only one message. The order of checks does not matter.)
    - If the playlist ID does not exist, redirect back to `/playlists/` with the message 'Invalid playlist ID' and an alert type of danger.
    - If the new playlist name is empty, redirect back to `/playlists/` with the message 'Playlist name cannot be empty' and an alert type of danger.
  - **Success:** On successful renaming, redirect back to `/playlists/` with the message 'Playlist renamed successfully' and an alert type of success.
3. **Delete Playlist:** Users can delete existing playlists (which should also remove associated entries from the `PlaylistTrack` table).
  - **Route:** `/playlist/delete/`





- **Function:** `delete_playlist` - It must validate that the playlist exists before deletion. Remember to also delete any associated records in the `PlaylistTrack` table to maintain referential integrity.
  - **Error:** If the playlist ID does not exist, redirect back to `/playlists/` with the message 'Invalid playlist ID' and an alert type of danger.
  - **Success:** On successful deletion, redirect back to `/playlists/` with the message 'Playlist deleted successfully' and an alert type of success.
4. **Add Tracks by Genre:** Users can add all tracks of a specific genre to a selected playlist. Tracks that are already in the playlist should not be duplicated.
- **Route:** `/playlist/add_tracks/`
  - **Function:** `add_tracks_by_genre` - It must validate that both the playlist and genre exist in the database before adding tracks. When adding tracks, ensure that duplicates are not created in the `PlaylistTrack` table. There is no need to report tracks that were skipped due to duplication.
  - **Errors:** (If multiple errors occur, display only one message. The order of checks does not matter.)
    - If the playlist ID does not exist, redirect back to `/playlists/` with the message 'Invalid playlist ID' and an alert type of danger.
    - If the genre ID does not exist, redirect back to `/playlists/` with the message 'Invalid genre ID' and an alert type of danger.
  - **Success:** On successful addition of tracks, redirect back to `/playlists/` with the message 'Tracks added successfully' and an alert type of success.
5. **Remove Tracks by Genre:** Users can remove all tracks of a specific genre from a selected playlist.
- **Route:** `/playlist/remove_tracks/`
  - **Function:** `remove_tracks_by_genre` - It must validate that both the playlist and genre exist in the database before removing tracks.
  - **Errors:** (If multiple errors occur, display only one message. The order of checks does not matter.)
    - If the playlist ID does not exist, redirect back to `/playlists/` with the message 'Invalid playlist ID' and an alert type of danger.
    - If the genre ID does not exist, redirect back to `/playlists/` with the message 'Invalid genre ID' and an alert type of danger.
  - **Success:** On successful removal of tracks, redirect back to `/playlists/` with the message 'Tracks removed successfully' and an alert type of success. All error and success messages should be displayed using the flash function in Flask. If any other error occurs during these operations, redirect back to `/playlists/` with the message 'An error occurred' and an alert type of danger. Remember you must add the appropriate decorators to each function to define the routes.

### 2.3.2 Evaluation Procedure

When testing your solution, the iMusic team will:

1. Navigate to the `/playlists/` route on the website.
2. Verify that all existing playlists and genres are accurately displayed.
3. Create a new playlist with a valid name and verify it appears in the list.
4. Rename an existing playlist and verify the name change is reflected.



5. Delete a playlist and verify it is removed from the list (and associated tracks are removed from `PlaylistTrack`).
6. Add tracks of a specific genre to a playlist and verify that the tracks are added correctly without duplicates.
7. Remove tracks of a specific genre from a playlist and verify that the tracks are removed correctly.

The iMusic team will repeat the above operations with various playlists and genres to ensure robustness. To further test your solution, the iMusic team will attempt operations with invalid playlist IDs, empty playlist names, and invalid genres to verify that appropriate error messages are displayed. Ensure that your solution can handle any valid data in the database, not just the examples provided. You are encouraged to test your solution with different data to ensure it works correctly—you will need to create your own test data for this task. The iMusic team will not provide you with additional test data.

### 2.3.3 Constraints

Your implementation must meet the following requirements:

- The entire implementation must be contained within the `playlists`, `get_all_genres` (from previous tasks), `get_all_playlists`, `create_playlist`, `rename_playlist`, `delete_playlist`, `remove_tracks_by_genre`, and `add_tracks_by_genre` functions. Do not create additional functions or modify the template file.
- The implementation must run as a Flask application, using the command `python iMusic.py` or `python3 iMusic.py`.
- Use the `sqlite3` module exclusively for database interactions. Other modules or approaches (e.g., `SQLAlchemy`) are not permitted.
- Do not modify the provided template file (`templates/playlists.html`).

### 2.3.4 Marking Scheme (12 Marks)

Task 3 is worth 12 of the 25 marks available for the assignment. The following criteria will be used to assess your work:

| ID    | Requirement                  | Details                                                                                                                                                       | Marks |
|-------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| RQ3_1 | Playlist and Genre Display   | Correct implementation of <code>get_all_playlists</code> to retrieve all playlists and use of <code>get_all_genres</code> to display genres in the template.  | 2     |
| RQ3_2 | Playlist Creation            | Correct implementation of <code>create_playlist</code> to successfully insert new (empty) playlists into the database upon valid submission.                  | 2     |
| RQ3_3 | Playlist Rename/Deletion     | Correct implementation of <code>rename_playlist</code> and <code>delete_playlist</code> , ensuring data consistency and referential integrity are maintained. | 2     |
| RQ3_4 | Genre-based Track Management | Correct implementation of <code>add_tracks_by_genre</code> and <code>remove_tracks_by_genre</code> , including logic to prevent duplicate entries.            | 2     |



| ID    | Requirement                   | Details                                                                                                                                   | Marks |
|-------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-------|
| RQ3_5 | Error and Success Handling    | Correct handling of application states, providing appropriate user feedback (flash messages) and redirection for both success and errors. | 2     |
| RQ3_6 | Security and Input Validation | Implementation of security measures to prevent SQL injection and appropriate validation to ensure database integrity.                     | 2     |

Table 4: Marking Criteria for Task 3.



## 3 Submission

To complete the assignment, submit an updated `iMusic.py` file containing your solutions to the tasks. No other files should be included. Ensure that your code is well-commented, and include your name and student ID at the top of the file, along with links to any resources you used (e.g., documentation, tutorials) to help you complete the assignment. Be sure to modify only the functions specified in the assignment brief. Avoid adding additional functions or modifying the provided template files. Note that you may create a database connection helper function within `iMusic.py` if you find it useful, but this is not required. If you choose to do so, please name it `get_db_connection`. Submit your solution via Moodle by the deadline - 22 December 2025 at 15:00.

### 3.1 Penalties

---

- **Late Submission:** Standard University policy - 5% penalty per working day late.
- **Incorrect File Name:** Submitting a file with an incorrect name will result in a 10% deduction from your final mark.
- **Use of External Libraries:** Only the `sqlite3`, `csv`, `flask` and the python standard library modules are permitted. Use of any other libraries will result in no marks awarded for that functionality. This may also cause runtime errors during testing by the iMusic team, leading to a significant mark deduction - most likely a zero for the assignment, as the solution likely will not run.

### 3.2 Academic Integrity

---

By submitting your work for assessment, you confirm that it is your own. Please familiarise yourself with the [Academic Misconduct policy](#). Remember:

- **Code Sharing:** Do not share your code or specific approach to solving the task with others, particularly in shared living spaces.
- **Code References:** Reference any external code or resources used, and ensure you understand and can explain any referenced material.
- **Protect Your Work:** Keep your work secure and regularly backed up, and do not allow others access to your files or computer. Be mindful of security in shared spaces.
- **Purpose of Assignment:** This assignment is designed to assess your understanding and application of course material. If you have questions about academic misconduct, please contact the module convenor.
- **Artificial Intelligence Tools:** You should not use AI tools (e.g., ChatGPT, GitHub Copilot) to generate code for this assignment. We've discussed the use of such tools in lectures - we're not opposed to their use in general, but for the learning objectives of this coursework, you should write your own code. If you do use AI tools, you must clearly document which parts of your code were generated by such tools, and you will not receive credit for those parts. Suspicious submissions will be followed up via oral examination which may impact your final grade if you cannot adequately explain your code.



Remember, your goal is to learn - not simply to get a grade. If you need support, make use of lab sessions and office hours.

## Acknowledgements

The data used in this assignment is based on the [Chinook Database](https://github.com/lerocha/chinook-database)<sup>1</sup>. Please note that the data provided in the database was not generated by the module team and may not be accurate or complete. The iMusic scenario is entirely fictional and does not represent a real company. We'd like to thank Jane Zhao, Huimin Tang, and Linjing Sun for their help reviewing this assignment.

---

<sup>1</sup>Chinook Database: <https://github.com/lerocha/chinook-database>